

OOP trong Java không hề khô khan: Giải mã 4 "trụ cột" qua các hệ thống thực tế



1. Lời mở đầu (Hook & Intro)

- **Vấn đề:** Người mới học Java thường bị ngợp bởi mớ lý thuyết OOP trừu tượng, dẫn đến việc viết code Java nhưng tư duy vẫn là lập trình thủ tục (như C).
- **Giải pháp:** Bỏ qua các định nghĩa sách vở. Bài viết này sẽ giúp bạn hiểu OOP qua lăng kính của một kỹ sư đang thiết kế các hệ thống thực tế.
- **Mục tiêu:** Nắm vững Đóng gói, Kế thừa, Đa hình, Trừu tượng để code gọn gàng, dễ bảo trì và dễ mở rộng.



2. Tính Đóng gói (Encapsulation): Chuyện cái két sắt trong ứng dụng đấu giá

- **Ví dụ thực tế:** Hãy tưởng tượng bạn đang viết một ứng dụng đấu giá trực tuyến. Mỗi người dùng có một cái "Ví tiền" (Wallet). Bạn tuyệt đối không được để số dư (**balance**) lơ lửng ra ngoài cho bất kỳ ai cũng có thể sửa thành 1 tỷ.
- **Áp dụng vào code Java:**
 - Biến **balance** phải được giấu kín (khai báo **private**).
 - Người ngoài (các class khác) muốn thay đổi số dư thì phải đi qua một cái "cửa kiểm duyệt" là các phương thức **public** (như **nạp_tiền()**, **đặt_giá()**).
 - Trong phương thức **đặt_giá()**, chúng ta viết logic kiểm tra: nếu số tiền đặt cược lớn hơn số dư đang có thì báo lỗi, cấm giao dịch.
- **Chốt lại:** Đóng gói là bảo vệ dữ liệu nội bộ khỏi những can thiệp không hợp lệ từ bên ngoài.

3. Tính Kế thừa (Inheritance): Tái sử dụng để không phải làm lại từ đầu

- **Ví dụ thực tế:** Vẫn trong hệ thống đấu giá đó, bạn có nhiều loại người dùng: **NgườiMua** (Bidder) và **NgườiBan** (Seller). Thay vì tạo hai bản thiết kế giống hệt nhau với các thông tin như Tên, Email, Mật khẩu.
- **Áp dụng vào code Java:**
 - Tạo ra một class cha (Base class) là **User** chứa các thuộc tính chung (Tên, Email) và hành vi chung (Đăng nhập, Đăng xuất).
 - Cho **Seller** và **Bidder** kế thừa (**extends**) từ **User**.

- Bây giờ, **Seller** chỉ cần viết thêm hàm **đăng_sản_phẩm()**, còn **Bidder** chỉ cần viết thêm hàm **đấu_giá()**. Phần chung đã được "hưởng xái" từ class cha.
- **Chốt lại:** Kế thừa giúp loại bỏ code lặp lại (DRY - Don't Repeat Yourself) và tạo ra một hệ thống phân cấp logic.

4. Tính Đa hình (Polymorphism): Một "lệnh", nhiều cách "cà thẻ"

- **Vấn đề thực tế:** Tưởng tượng bạn đang code module thanh toán (Checkout) cho một ứng dụng đấu giá trực tuyến. Khi người dùng thắng một phiên đấu giá, họ cần trả tiền. Nhưng đời không như mơ, khách hàng mỗi người một ý: người thích xài ví Momo, người quen quét mã VNPAY, người lại chuộng mở app ngân hàng ra chuyển khoản thẳng.
- Nếu code theo kiểu "cổ đại" (thủ tục), bạn sẽ phải viết một núi các câu lệnh **if-else**: *Nếu là Momo thì gọi API Momo, nếu là ngân hàng thì sinh mã QR...* Code sẽ dài ngoằng và cực kỳ dễ toang mỗi khi cần thêm cổng thanh toán mới.
- **Cách OOP giải quyết (Áp dụng Đa hình):** Chúng ta tạo ra một bản hợp đồng chung (Interface) tên là **PaymentMethod**. Hệ thống cốt lõi chỉ cần biết: "Mày là một phương thức thanh toán, mày phải có hàm trừ tiền!". Dưới đây là đoạn code thực tế:

```

1 // 1. Tạo một Interface chung cho mọi phương thức thanh toán
2 public interface PaymentMethod { 3 usages 2 implementations
3     void processPayment(double amount); 1 usage 2 implementations
4 }
5 // 2. Các class con tự định nghĩa cách thanh toán của riêng mình
6 public class MomoPayment implements PaymentMethod { no usages
7     @Override 1 usage
8     public void processPayment(double amount) {
9         System.out.println("Đang tạo token kết nối API Momo...");
10        System.out.println("Thanh toán " + amount + " VND qua ví Momo thành công!");
11    }
12 }
13 public class BankTransferPayment implements PaymentMethod { 1 usage
14     @Override 1 usage
15     public void processPayment(double amount) {
16         System.out.println("Đang khởi tạo mã QR chuyển khoản tới VPBank...");
17         System.out.println("Giao dịch " + amount + " VND hoàn tất qua biển động số dư.");
18    }
19 }

```

```

20 // 3. Hệ thống xử lý (Client code)
21 public class CheckoutSystem {
22     public static void main(String[] args) {
23         // Đa hình thể hiện rõ nhất ở đây:
24         // Khai báo kiểu cha (PaymentMethod) nhưng khởi tạo đối tượng con (BankTransferPayment)
25         PaymentMethod userPayment = new BankTransferPayment();
26         // Hệ thống chỉ cần gọi ĐÚNG 1 câu lệnh này.
27         // Java sẽ tự động "nhận diện" biến userPayment đang trỏ tới đối tượng nào
28         // để thực thi đúng logic của ngân hàng hay Momo.
29         userPayment.processPayment( amount: 1500000);
30     }
31 }

```

5. Tính Trừu tượng (Abstraction): Chỉ quan tâm "Làm gì", giấu đi "Làm như thế nào"

- **Ví dụ thực tế:** Nhìn lại cái giao diện trên điện thoại của bộ Central Hub nhà thông minh. Trên màn hình có cái nút "Bật". Bạn ấn vào, thiết bị chạy. Bạn có cần biết dòng điện chạy qua bảng mạch điện tử dưới lớp vỏ nhựa như thế nào không? Không! Bạn chỉ cần biết dùng cái nút.
- **Áp dụng vào code Java:**
 - Sử dụng `interface` hoặc `abstract class`.
 - Tạo một `interface SmartDevice` chỉ định nghĩa một hợp đồng rỗng: `void turnOn();`.
 - Người viết giao diện (UI) chỉ quan tâm: "Tôi biết cái cục này là SmartDevice, nó chắc chắn có hàm `turnOn()`, tôi cứ gọi nó thôi". Các kỹ sư phần cứng phía sau sẽ phải tự đi viết chi tiết ruột gan của cái hàm đó.
- **Chốt lại:** Trừu tượng giúp giảm bớt sự phức tạp cho người sử dụng (người gọi code), tập trung vào thiết kế tính năng thay vì sa đà vào chi tiết cài đặt ngay từ đầu.

6. Kết luận & Call to Action

- **Tổng kết:** OOP không phải là lý thuyết ép buộc, nó là công cụ để mô hình hóa thế giới thực vào phần mềm một cách tự nhiên nhất.
- **Kêu gọi:** Thử mở IDE lên, bắt tay vào viết một dự án nhỏ (như quản lý nhà thông minh) để thấy 4 tính chất này phối hợp với nhau mượt mà như thế nào nhé.